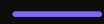


TURI LABS — OPERATOR SERIES

The Operator's Edge

How to get 10x more out of Claude Code and Codex —
without switching tools.



*"The tool is 20% of the result.
The operator is 80%."*

Yashwanth Sai

01

The Operator Mindset

Most people blame the model when things go wrong — bad output, half-finished tasks, ignored instructions. But the model is rarely the problem. The operator is.

01 No plan, just vibes

You describe a feature loosely and let the agent run. It builds something that kind of works. But the architecture is a mess, the tests are wrong, and three sessions later you're refactoring everything. This happens because you skipped the most important step — thinking before prompting.

02 Infinite context

The million token context window sounds like a superpower. It is actually a trap. The longer your session runs, the more the agent loses the thread. Instructions get ignored. Earlier decisions get forgotten. Good operators keep sessions short and focused.

03 No review loop

You let the agent finish, skim the output, and move on. This compounds over time. Small errors become structural problems. Tests get silently rewritten to match broken behavior. Code that "passes" is not code that works.

THE MINDSET SHIFT

Treat the agent like a fast, capable junior engineer. Smart enough to execute, but needs clear direction, tight scope, and regular check-ins. Your job is not to prompt — it is to lead.

02

The Operator Stack

Four things that separate operators who get real output from operators who spin in circles.

01

Always plan before you code

Before any coding session, enter plan mode. Write a scoped, specific prompt. Not "build me a login system" — but exactly what model, what constraints, what patterns to follow. The more specific your plan, the less the agent improvises. And improvisation is where things break. Review the plan before approving it.

02

Set up your CLAUDE.md or AGENTS.md properly

This file is your standing instructions. The agent reads it every session. Most people leave it empty. That is a wasted opportunity. Include:

- Your git workflow — branch naming, commit format, never push to main
- Testing rules — if tests break, stop and ask, never fix them silently
- File size and structure preferences
- Non-obvious conventions in your codebase

Keep it under 100 lines. Concise instructions get followed. Walls of text get ignored.

03

Context hygiene

Start a new session for each distinct task. Do not carry yesterday's debugging session into today's feature work. The agent does not need the full history — it needs the right context for the task at hand. If a session runs long and the agent starts doing strange things, that is your signal. Close it, summarize what was done, start fresh.

04

Never let the agent modify tests to make them pass

This is the most dangerous habit agents have. When a test fails, the path of least resistance is to change the test. Your rule: if tests break after a change, stop the agent. Review what broke and

why. Only then decide whether the test or the implementation needs to change. You make that call — not the agent.
